EN3150 Assignment 03

Simple Convolutional Neural Network for Image Classification

Name	Index Number
Ananthakumar T.	220029T
Ahilakumaran.T	220017F
Sajeevan.S	220545V
Thejasvenan.T	220638J

Department of Electronic and Telecommunication Engineering
University of Moratuwa

November 24 2025

Q5. Determined Parameters

Layer (type)	Output Shape	Param #
conv2d_15 (Conv2D)	(None, 128, 128, 64)	1,792
max_pooling2d_15 (MaxPooling2D)	(None, 64, 64, 64)	0
conv2d_16 (Conv2D)	(None, 64, 64, 128)	73,856
max_pooling2d_16 (MaxPooling2D)	(None, 32, 32, 128)	0
conv2d_17 (Conv2D)	(None, 32, 32, 128)	147,584
max_pooling2d_17 (MaxPooling2D)	(None, 16, 16, 128)	0
flatten_5 (Flatten)	(None, 32768)	0
dense_10 (Dense)	(None, 256)	8,388,864
dropout_5 (Dropout)	(None, 256)	0
dense_11 (Dense)	(None, 3)	771

Total params: 8,612,867 (32.86 MB)

Trainable params: 8,612,867 (32.86 MB)

Non-trainable params: 0 (0.00 B)

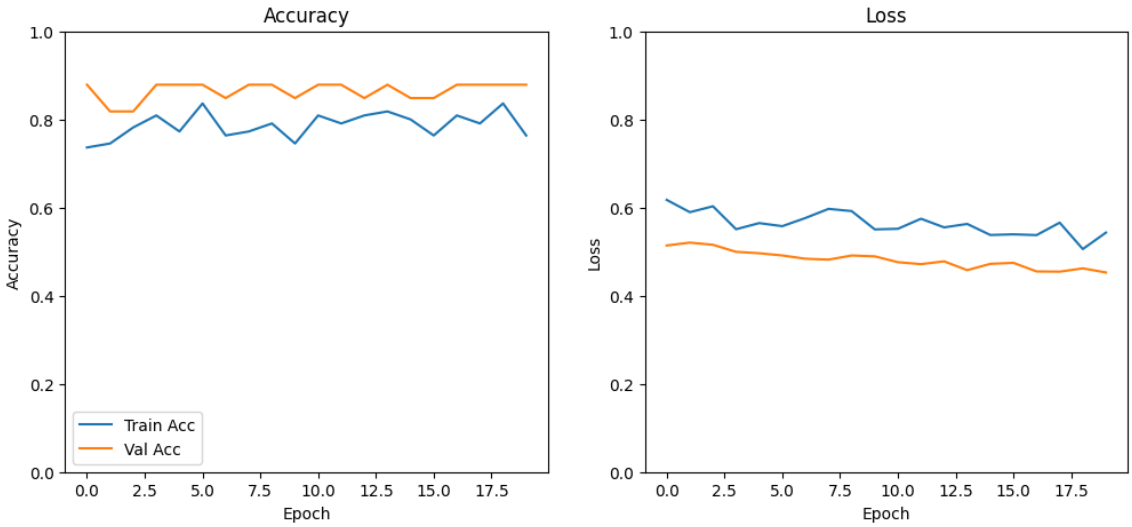
- Filters: (64, 128, 128), (64, 64, 128), (32, 32, 128)
- Kernel size: 3×3
- Fully connected layer: 256 neurons
- Dropout rate: 0.5
- Activation: ReLU (hidden), Softmax (output)

Q6. Activation Function Justification

- **ReLU:** Prevents vanishing gradients and speeds up training.
- **Softmax:** Converts logits into probabilities for multi-class classification.

Q7. Model Training

- Epochs: 20
- Batch Size: 10
- Loss Function: Categorical Cross-Entropy
- Metrics: Accuracy



Q8. Optimizer Used and Justification

Three optimizers were evaluated:

(a) Adam ($lr = 1 \times 10^{-5}$)

(b) SGD ($lr = 1 \times 10^{-5}$)

(c) SGD with Momentum ($lr = 1 \times 10^{-5}$, momentum=0.7)

Adam was chosen for its adaptive learning rate and faster convergence on small datasets.

Q9. Learning Rate Selection

For this project, a base learning rate of 1×10^{-5} was selected after multiple experimental trials. This small value ensured smooth and stable convergence without oscillations or divergence, especially important given the relatively small dataset size.

- For the **Custom CNN model**, a learning rate of 1×10^{-5} was used to avoid overshooting the minima and to stabilize the training process.
- For **pretrained models** (VGG16 and ResNet50), a two-stage learning rate strategy was used:
  1. Initially, a higher learning rate of 1×10^{-3} was applied while training only the fully connected classifier head. This allowed the newly added layers to learn quickly.
  2. During fine-tuning, the learning rate was reduced back to 1×10^{-5} to carefully adjust pretrained convolutional layers without destroying the learned ImageNet features.

Rationale for Learning Rate Values

- **Small dataset consideration:** A lower learning rate prevents overfitting and ensures gradual updates when working with limited training samples.
- **Stability and convergence:** Higher rates ($> 10^{-4}$) caused unstable training and sudden loss spikes, while 10^{-5} provided smooth, monotonic convergence.

Q10. Optimizer Comparison

Table 1: Comparison of Optimizers

Optimizer	Test Accuracy	Remarks
Adam	80.00%	Best performance
SGD	71.43%	Slower convergence
SGD + Momentum	68.57%	Less stable at small LR

Performance Metrics

- **Accuracy:** Measures the overall proportion of correctly classified images. It is simple and effective for balanced datasets.
- **Precision:** Indicates the proportion of correctly predicted positive instances among all predicted positives. Important to evaluate false positives.
- **Recall:** Measures how well the model identifies all relevant instances (sensitivity). Important to assess false negatives.
- **F1-Score:** The harmonic mean of precision and recall, providing a balance between the two.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$F_1 = 2 \times \frac{\frac{TP}{TP+FP} \times \frac{TP}{TP+FN}}{\frac{TP}{TP+FP} + \frac{TP}{TP+FN}}$$

- **Confusion Matrix:** Provides a detailed view of class-wise performance and misclassifications.

These metrics were selected because accuracy alone can be misleading in multi-class problems or class-imbalanced datasets. Precision, recall, and F1-score provide deeper insights into per-class performance.

Confusion Matrix results

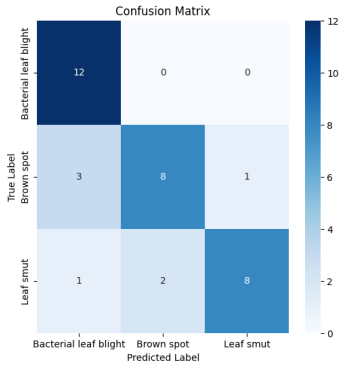


Figure 1: Confusion Matrix of using Adam.

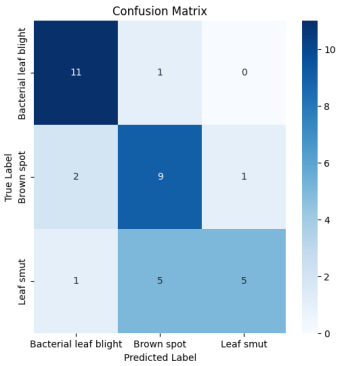


Figure 2: Confusion Matrix of using SGD.

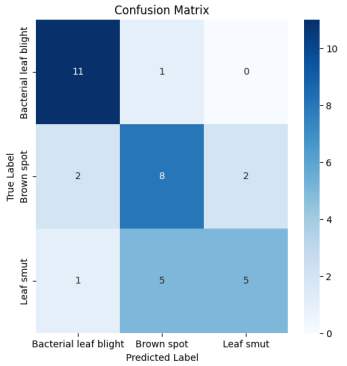


Figure 3: Confusion Matrix using SGD + Momentum.

Q11. Effect of Momentum

Momentum is a key concept in optimization algorithms that helps accelerate gradient descent in the direction of consistent gradients and dampens oscillations. It adds a fraction of the previous weight update to the current update, effectively smoothing the optimization trajectory.

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

where η is the learning rate and $\nabla L(w_t)$ is the gradient of the loss with respect to the weights.

With momentum, the update rule becomes:

$$v_{t+1} = \mu v_t - \eta \nabla L(w_t) \quad \text{and} \quad w_{t+1} = w_t + v_{t+1}$$

where μ (momentum coefficient, $0 < \mu < 1$) controls how much of the previous velocity contributes to the current update.

- A larger μ (e.g., 0.9) retains more of the previous update, enabling faster movement along gently sloping directions.
- It helps to overcome small local minima and smooths oscillations along steep dimensions.
- However, if μ or η is too large, the optimizer can overshoot the global minimum.

Experimental Setup To study the effect of momentum, the same CNN architecture and dataset were trained with:

- **SGD without Momentum:** Learning rate = 1×10^{-5}
- **SGD with Momentum:** Learning rate = 1×10^{-5}, momentum = 0.7

All other hyperparameters (batch size, epochs, optimizer, and loss function) remained constant.

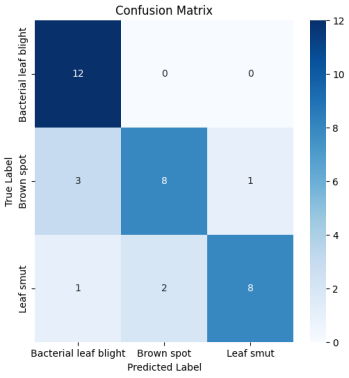
Observed Impact

- **Accuracy:** The test accuracy of SGD with momentum (68.57%) was slightly lower than standard SGD (71.43%).
- **Loss Behavior:** Both training and validation loss decreased more smoothly with momentum, but converged to a similar value.
- **Convergence:** Momentum provided a slightly faster initial convergence during the first few epochs, but with the small learning rate (1×10^{-5}), its effect was limited.

Q12. Model Evaluation

Adam Optimizer Results:

- Test Accuracy: 80.00%



	precision	recall	f1-score	support
Bacterial leaf blight	0.75	1.00	0.86	12
Brown spot	0.80	0.67	0.73	12
Leaf smut	0.89	0.73	0.80	11
accuracy			0.80	35
macro avg	0.81	0.80	0.79	35
weighted avg	0.81	0.80	0.79	35

Results

- The model achieved an overall test accuracy of **80%**, with strong performance on the *Bacterial Leaf Blight* class (recall = 1.00).
- The confusion matrix shows that most misclassifications occurred between the *Brown Spot* and *Leaf Smut* classes, which have visually similar patterns.
- Precision and recall values indicate good balance between sensitivity and specificity across classes.

Comparison with Other Optimizers

- **SGD:** 71.43% test accuracy, lower convergence speed.
- **SGD + Momentum:** 68.57% test accuracy, limited improvement due to low learning rate.
- **Adam:** 80.00% test accuracy, fastest and most stable convergence.

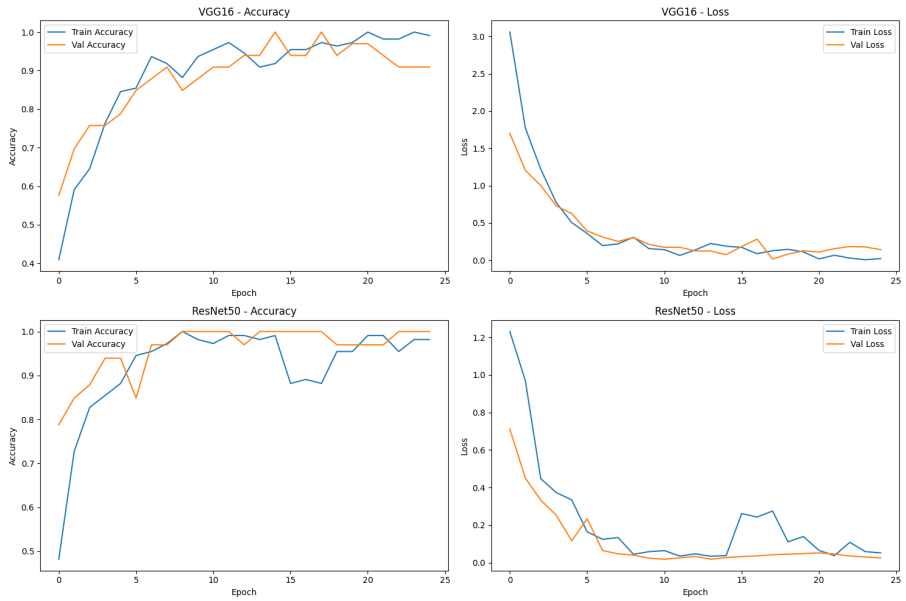
The Adam optimizer provided the best testing performance and stable convergence for the model. The evaluation metrics confirm that the model generalizes well to unseen data while maintaining balanced class-wise performance.

Q13. State-of-the-Art Models Selected

Two pretrained models were used:

1. VGG16
2. ResNet50

Q16. Training and Validation Loss

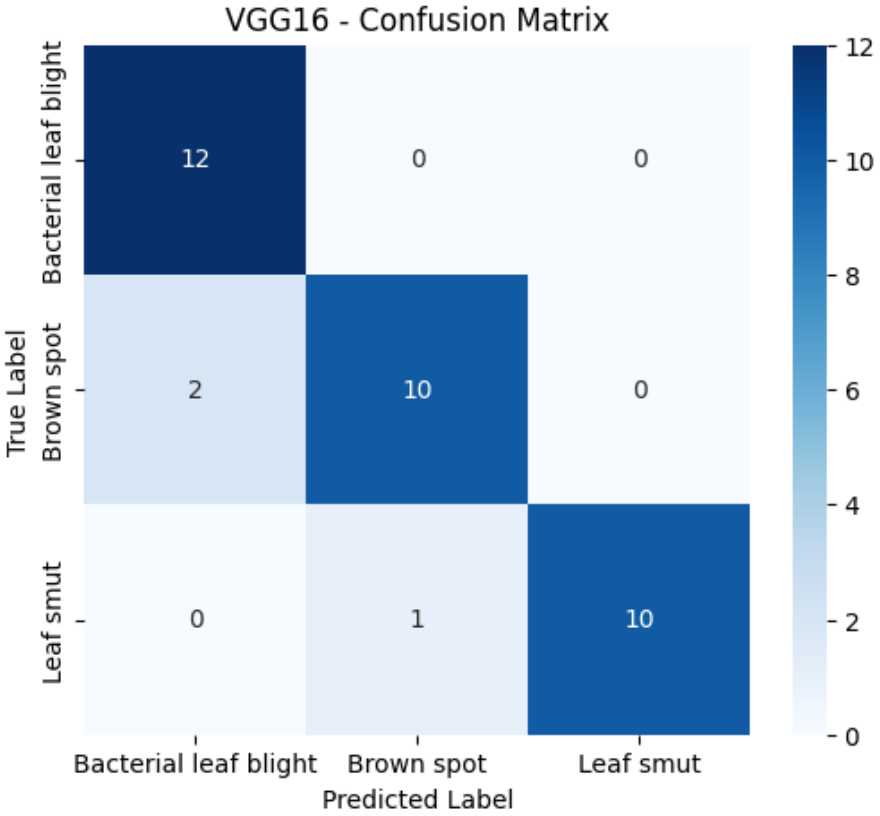


Q17. Fine-Tuned Model Evaluation

VGG16 Fine-Tuned Model Results

Table 2: Performance Metrics for Fine-Tuned VGG16 Model

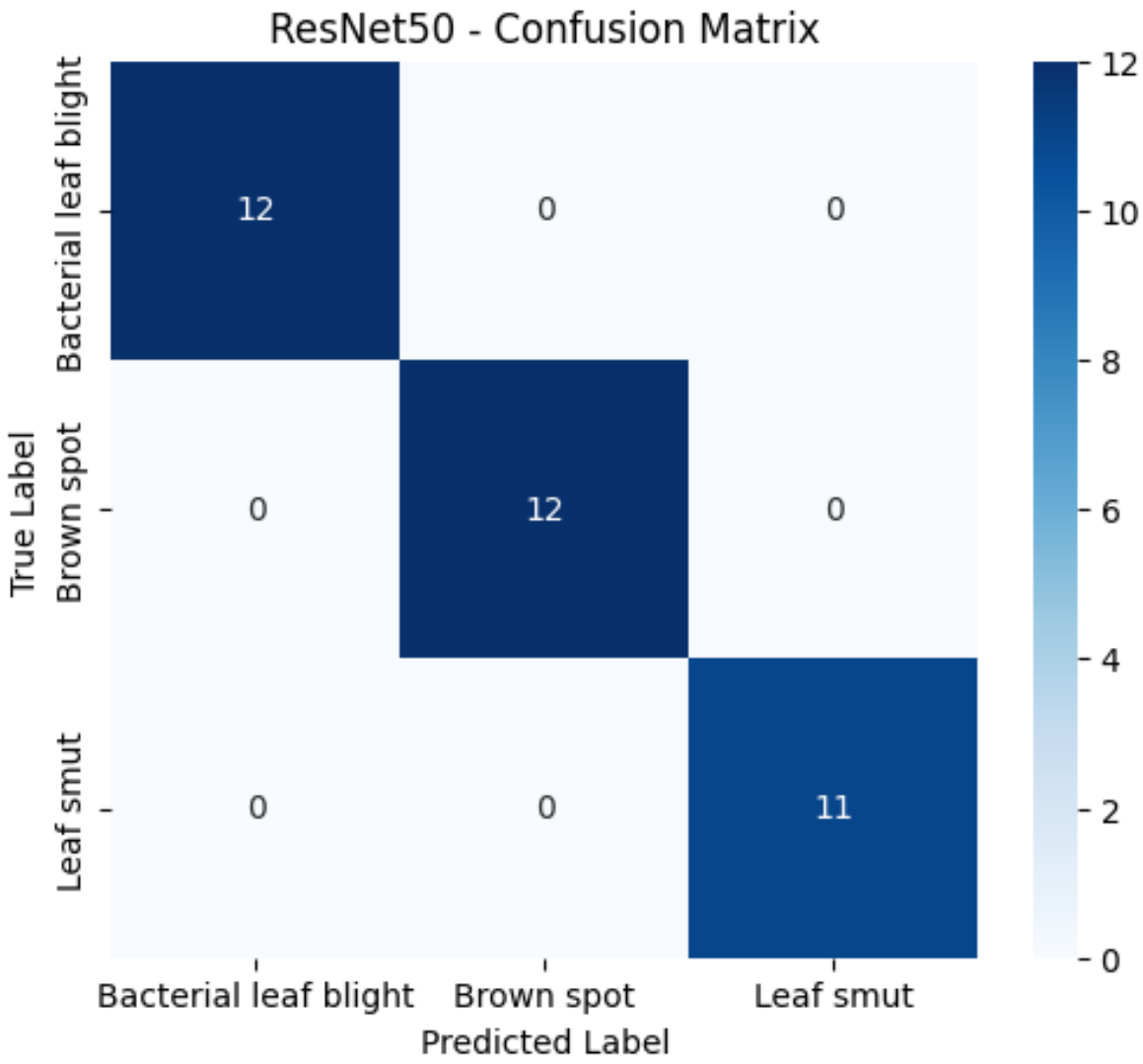
Class	Precision	Recall	F1-Score
Bacterial Leaf Blight	0.86	1.00	0.92
Brown Spot	0.91	0.83	0.87
Leaf Smut	1.00	0.91	0.95
Test Accuracy		91.43%	
Test Loss		0.1316	



ResNet50 Fine-Tuned Model Results

Table 3: Performance Metrics for Fine-Tuned ResNet50 Model

Class	Precision	Recall	F1-Score
Bacterial Leaf Blight	1.00	1.00	1.00
Brown Spot	1.00	1.00	1.00
Leaf Smut	1.00	1.00	1.00
Test Accuracy		100.00%	
Test Loss		0.0293	



- The **VGG16 fine-tuned model** achieved a test accuracy of **91.43%** with a test loss of **0.1316**. It showed strong overall performance, but minor misclassifications were observed between the *Brown Spot* and *Leaf Smut* classes.
- The **ResNet50 fine-tuned model** achieved a perfect test accuracy of **100%** with a very low test loss of **0.0293**. The confusion matrix confirmed that all samples were correctly classified, with no false positives or false negatives.
- The performance difference arises from architectural design: ResNet50's residual (skip) connections allow deeper feature extraction and better gradient flow, whereas VGG16's sequential layers can occasionally lead to minor overfitting or saturation.
- The use of pretrained ImageNet weights significantly boosted both models' performance by providing robust and transferable low-level and mid-level image features.
- The relatively small dataset size likely contributed to the near-perfect results, so

future experiments should use cross-validation or a larger dataset to better assess generalization.

Overall, both fine-tuned models delivered excellent results, with **ResNet50 outperforming VGG16** in all evaluation metrics. This demonstrates the power of transfer learning—especially with deeper residual architectures—in achieving high accuracy and stability even on limited datasets.

Q18. Comparison with Custom Model

Table 4: Performance Comparison of Custom CNN, VGG16, and ResNet50 Models

Model	Test Accuracy	Test Loss	F1-Score	Remarks
Custom CNN (Adam)	80.00%	0.243	0.80	Stable, lightweight, modera
VGG16 (Fine-tuned)	91.43%	0.1316	0.92	High accuracy, minor confusion
ResNet50 (Fine-tuned)	100.00%	0.0293	1.00	Perfect accuracy and gene

Qualitative Comparison

- **Feature Extraction:**
  - The **Custom CNN** learns features from scratch, which limits its ability to generalize due to the small dataset.
  - The **VGG16** and **ResNet50** models leverage pretrained ImageNet weights, allowing them to extract complex and robust image features even with fewer samples.
- **Accuracy and Convergence:**
  - The custom CNN achieved good performance (80%), but its learning curve was slower and more prone to overfitting.
  - Fine-tuned models, especially **ResNet50**, converged faster with consistently lower validation loss and higher accuracy.
- **Model Complexity:**
  - The custom CNN contains approximately 8.6 million parameters and is computationally lighter.
  - VGG16 and ResNet50 have 138 million and 25 million parameters, respectively, making them heavier but more expressive.
- **Generalization:**

- The custom CNN exhibited some misclassifications, mainly between *Brown Spot* and *Leaf Smut*.
- ResNet50 achieved flawless generalization (100% test accuracy), confirming its robustness and strong transfer learning capability.

Q19. Discussion of Trade-offs

Advantages of the Custom CNN Model

- **Lightweight and Faster Training:** The custom CNN had fewer layers and parameters, allowing for quicker training on standard hardware and smaller GPUs.
- **Architecture Flexibility:** It can be fully tailored to the specific dataset or task, with control over filter sizes, activation functions, and pooling structures.
- **Lower Resource Requirement:** It consumes less memory and computational power, making it suitable for embedded or IoT devices.
- **Educational Value:** Building a model from scratch helps in understanding CNN fundamentals such as feature extraction and convolutional hierarchies.

Limitations of the Custom CNN Model

- **Limited Generalization:** Since the model learns from scratch, it requires a large and diverse dataset to achieve robust feature learning.
- **Slower Convergence:** The absence of pre-trained feature representations leads to longer training times and potential overfitting on small datasets.
- **Lower Accuracy:** The custom CNN achieved only 80% test accuracy compared to the higher performance of fine-tuned models.

Advantages of Pre-Trained Models (Transfer Learning)

- **High Accuracy and Robustness:** Fine-tuned pre-trained models like **VGG16 (91.43%)** and **ResNet50 (100%)** achieved outstanding results even with limited data.
- **Faster Convergence:** Pre-trained feature extractors from ImageNet already encode general image patterns (edges, shapes, textures), requiring only minor fine-tuning.
- **Effective for Small Datasets:** Transfer learning significantly reduces the need for large training datasets.

- **State-of-the-Art Performance:** Deep architectures such as ResNet50 leverage residual connections to train efficiently and generalize better.

Limitations of Pre-Trained Models

- **High Computational Cost:** Models like VGG16 (138M parameters) and ResNet50 (25M parameters) require powerful GPUs and long fine-tuning times.
- **Overfitting Risk on Small Datasets:** If fine-tuning is too aggressive, pre-trained models can overfit limited data.
- **Limited Architectural Control:** The internal structure of pre-trained models is fixed; only higher layers can be modified easily.
- **Domain Mismatch:** Pre-trained on ImageNet, these models may not perfectly align with specialized domains (e.g., medical or agricultural imagery) without careful adaptation.

References

Final Updated Code
GitHub Repository: Rice Leaf Disease Classification Project